

CSIC30216

AI-in-the-Loop in Software Project Cycle

AI 導入軟體專案流程

Week 1: Introduction to AI-assisted
software engineering

Instructors:

Dennis Wang 王邦任

Yu-Chun (Grace) Yen 顏羽君

Date: 2026-09-07

Announcements

(before we get started)

W1 Announcements

加簽 / Class Auditing

- We are **AT CAPACITY**
 - Given the limitation of resource, we will **NOT** take any auditing request for this class
(本堂課程因為資源限制，並不開放加簽，希望同學們見諒)
 - If you are not enrolled, please kindly leave the seat for students who are
- The **official waitlist (候補)** is the only way for you to get into the class
 - If you are enrolled but have decided **not** to take this course, **please kindly drop early** so we can keep the list rolling
- **We expect to offer this course every fall from now on**, and hope that we will be able to see you the following year if you won't be able to get in this time

Welcome!

Learning Objectives For Today

- Describe the engineer's three responsibilities in the agentic era: specifying intent, evaluating trajectories, and knowing when to take back control
- Understand the course structure, components, expectations, and resources
- Identify at least one decision the model made without being asked during the demo and evaluate whether you would have made the same choice
- Write your first prompt engineering log entry based on today's lab

Learning Objectives For *The Semester*

- *Articulate and apply effective practices of AI-assisted engineering and prompt engineering in software development*
- *Understand core software engineering concepts that contribute to successful software development, and how to effectively and ethically incorporate AI in it*
- *Analyze the broader implications and societal impacts of AI-assisted engineering and its infrastructure*

The question this course is built around

- "How does this make me a better software engineer working with AI?"
 - Every lecture, lab, and assignment is designed to answer this question
- This course is *not about* how to use AI tools, or how AI works.
 - We will occasionally teach both of those, but in service of the core question

What you will be able to do by December

- *Specify* intent precisely enough for an AI agent to execute it reliably
- *Evaluate AI output* — not just "does it look right" but "is it actually correct and trustworthy"
- Know *When to take back control* — override, correct, or choose not to use AI output
- *Build, test, and document* AI-integrated software that other engineers can understand and maintain
- *Collaborate* on a shared codebase where AI is a first-class participant

Re-introduction to “*Vibe Coding*” — Where We Actually Are

“Vibe Coding”

- Andrej Karpathy (founding member of OpenAI, ex-Director of AI at Tesla, Now: Anthropic)
- The term was actually *coined only less than two years ago*.
- “*fully give in to the vibes, (...) forget that the code even exists*”

Andrej Karpathy, <https://x.com/karpathy/status/1886192184808149383>, X/Twitter, 2025-02-03



What 'Vibe Coding' Actually Means (in 2026)

- 1.5 years later....Vibe coding is largely adopted in production
- **90% of developers** regularly used **at least one AI tool at work for coding and development tasks** (January, 2026)
- Companies are reporting a growing percentage of their codebases are being generated by AI
 - **Google** (>30%, Q1 2025), **Microsoft** (20–30%, Q1 2025), **Anthropic** (70–90%, Q4 2025)

Note: these are self-reported, with definitions vary. They could be treat as direction, not precise measurement

Beatrice Nolan, <https://fortune.com/2026/01/29/100-percent-of-code-at-anthropic-and-openai-is-now-ai-written-boris-cherny-roon/>, 2026-01-29

Sherin Shibu, <https://www.entrepreneur.com/business-news/ai-is-taking-over-coding-at-microsoft-google-and-meta/490896>, 2025-04-30

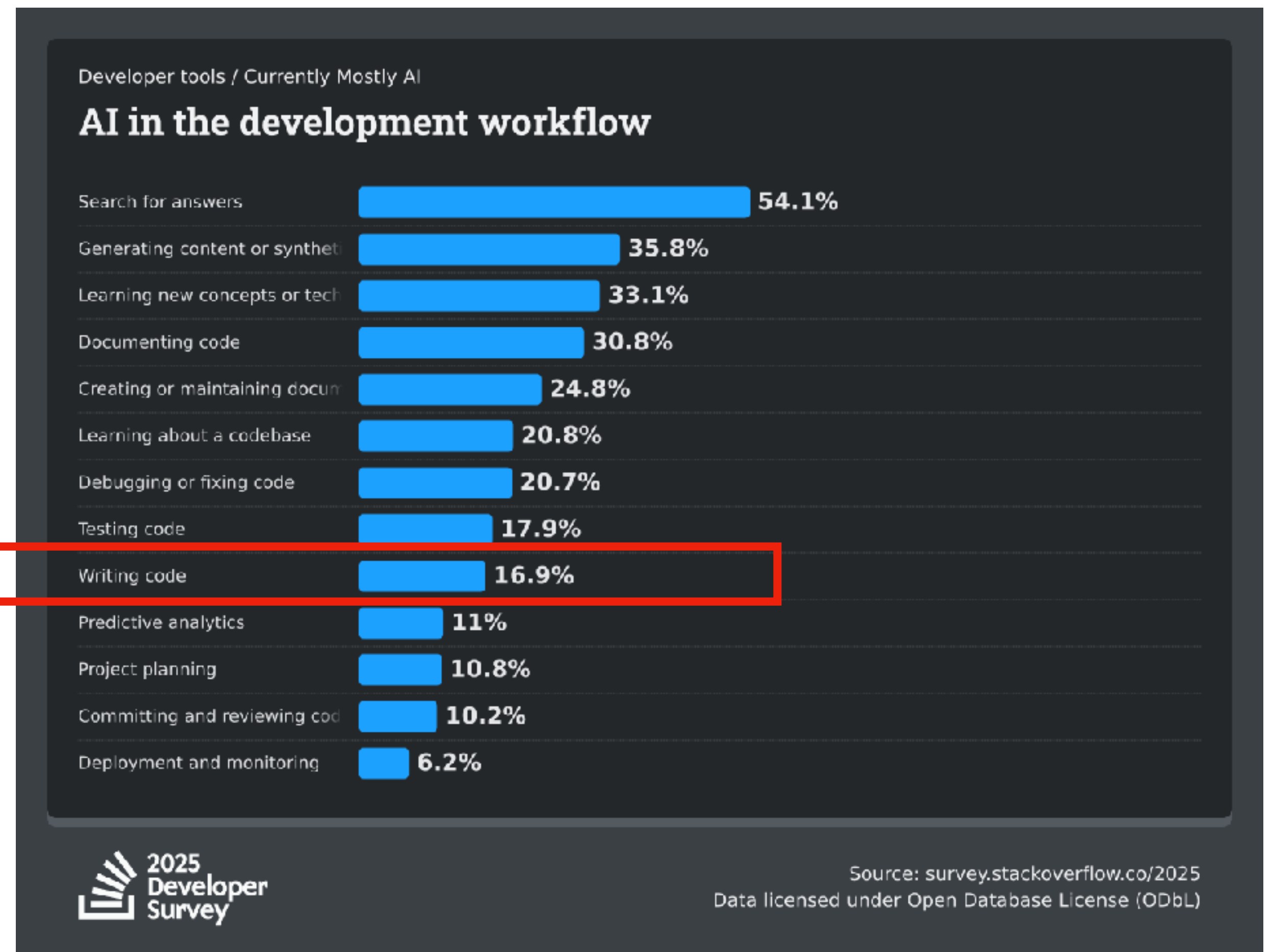
Jordan Novet, <https://www.cnbc.com/2025/04/29/satya-nadella-says-as-much-as-30percent-of-microsoft-code-is-written-by-ai.html>, 2025-04-29

Mikhail Bogdanov, <https://blog.jetbrains.com/research/2026/04/which-ai-coding-tools-do-developers-actually-use-at-work/>, 2026-04-02

What 'Vibe Coding' Actually Means (in 2026)

Have we “forget that the code even exists” yet?

- **No.** With the advancement of tools, we're seeing evidences that might suggest the opposite:
- Instead of just **generating the code**, AI are used **across the development process**
- Software engineering is **more than just writing codes**



What 'Vibe Coding' Actually Means (in 2026)

Have we “forget that the code even exists” yet?

- **74% of developers** had already adopted specialized AI tools for developers (e.g., coding assistants, editors, and agents) (January, 2026)
- Less about ignoring the code and more about a **fundamentally different interaction model**

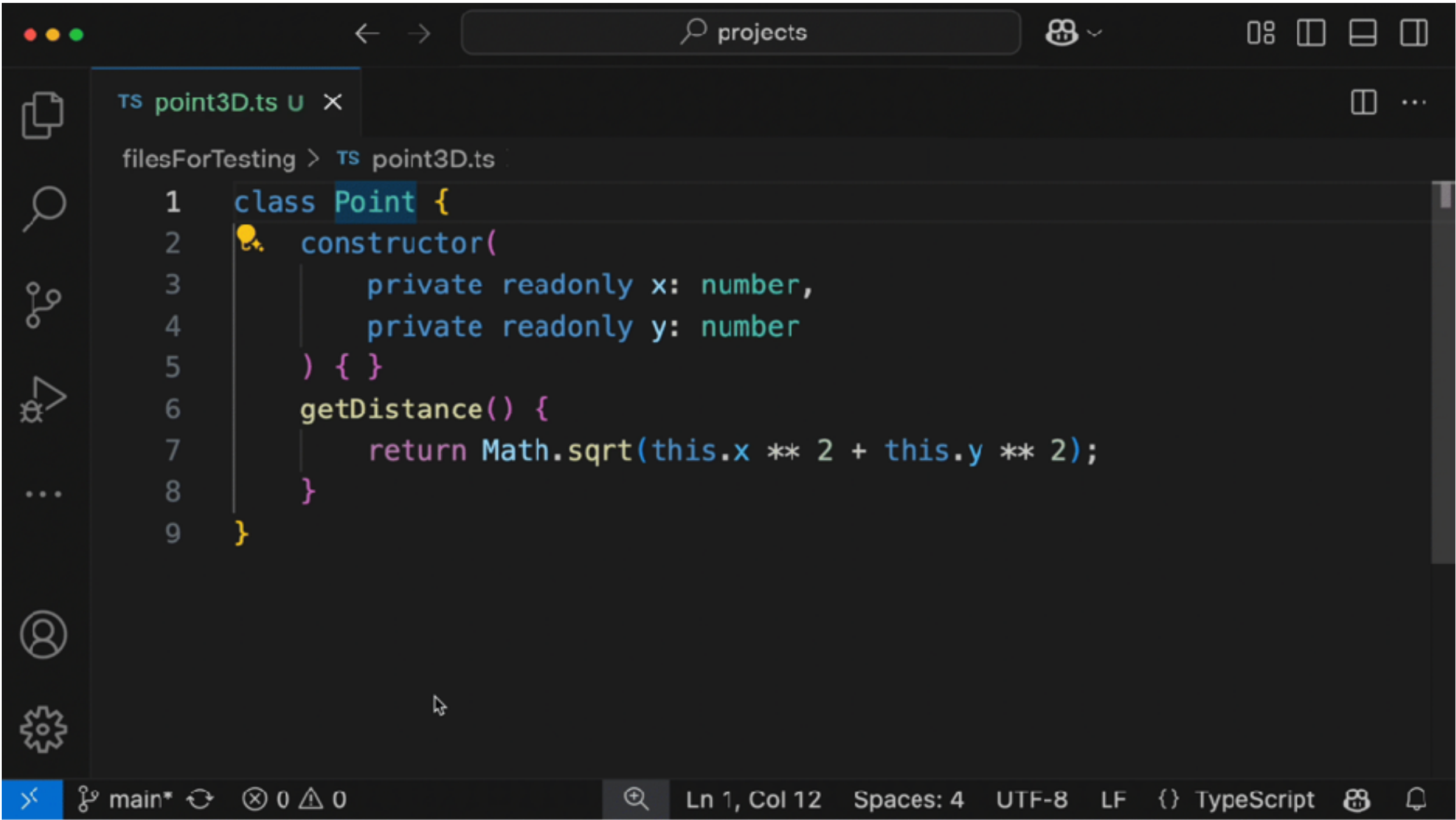
What 'Vibe Coding' Actually Means (in 2026)

- Let us take a look at what people are doing through looking at the spectrum of **AI-assisted development modes**
 - Option 1: Autocomplete / inline suggestion
 - Option 2: Chat-driven
 - Option 3: Agentic
- What differs between these? Not whether you prompt — you prompt in all three.
 - It is **who controls what the model sees**

The spectrum of AI-assisted development

Option 1: Autocomplete / inline suggestion. Model fills in as you type

	Option 1: Autocomplete
What the model sees	Nearby lines, open tabs
What you must specify	Nothing. You just type
Who assembles the context	Your cursor, implicitly
Where it fails	Anything not locally visible
What it's good at	Mechanical, pattern-based completions

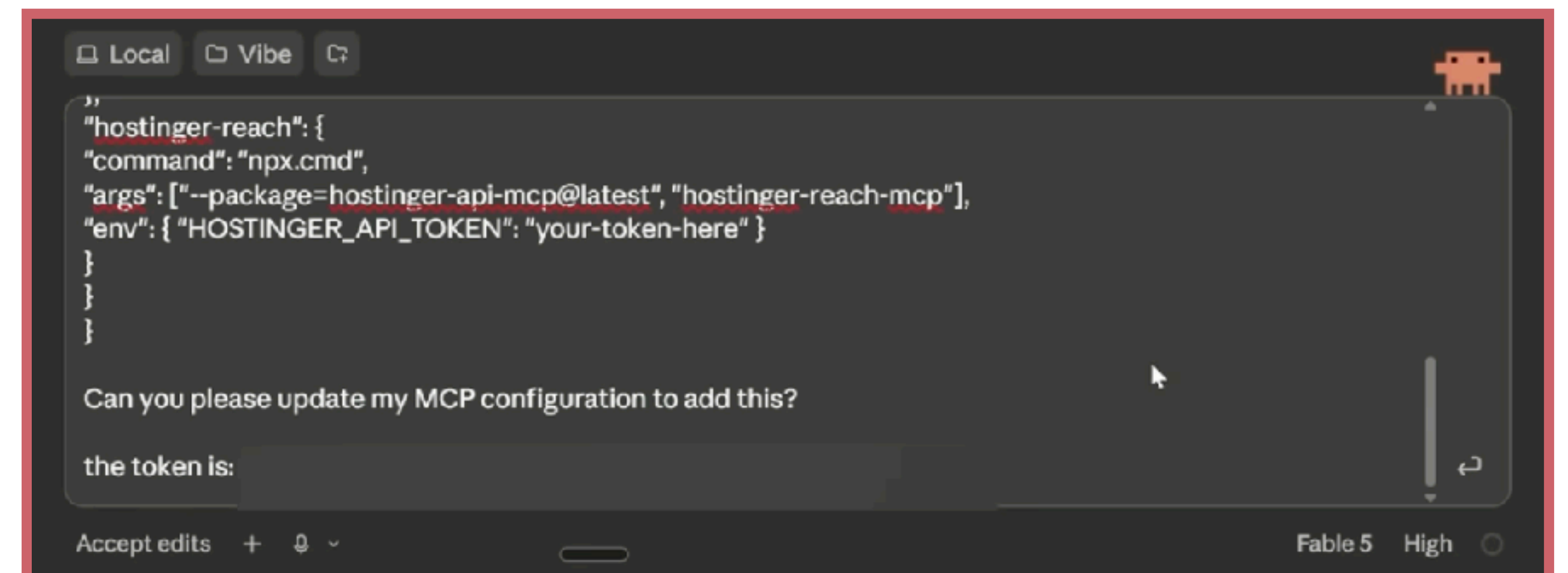
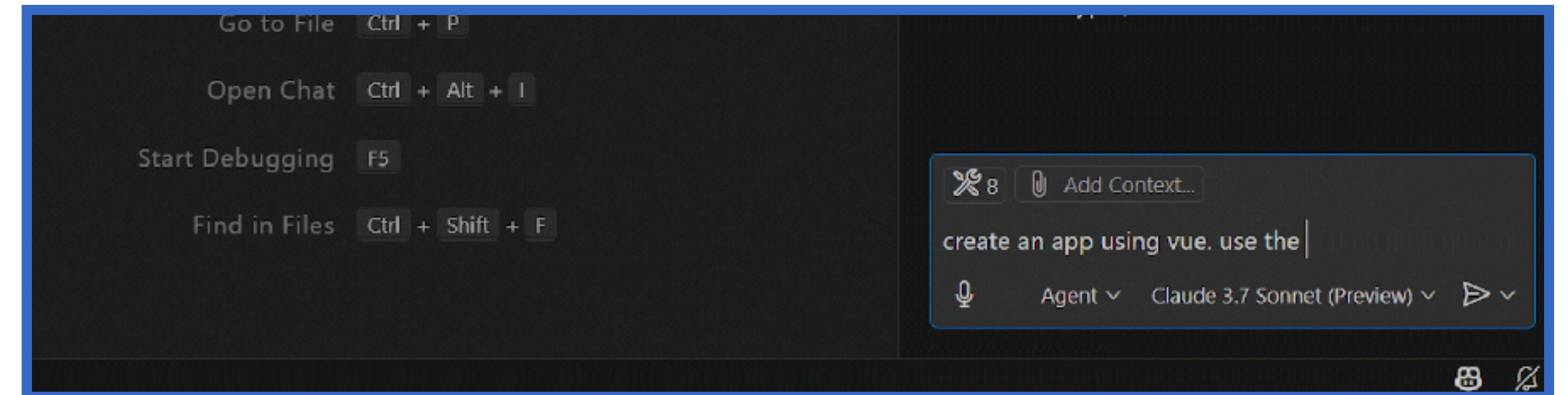
A screenshot of the Visual Studio Code editor interface. The main editor window shows a TypeScript file named 'point3D.ts'. The code defines a 'Point' class with a constructor and a 'getDistance()' method. A yellow dot in the constructor parameter list indicates a suggestion point. The sidebar on the left shows the Explorer, Search, and Run and Debug views. The bottom status bar shows 'Ln 1, Col 12', 'Spaces: 4', 'UTF-8', 'LF', and 'TypeScript'.

GitHub Copilot Extension for VS Code

The spectrum of AI-assisted development

Option 2: Chat-driven. You describe a task, model responds, you iterate

	Option 2: Chat-driven
What the model sees	Exactly what you paste
What you must specify	Everything the model needs
Who assembles the context	You, by pasting
Where it fails	Tasks needing broad codebase context
What it's good at	Tasks that you can specify clearly in the window

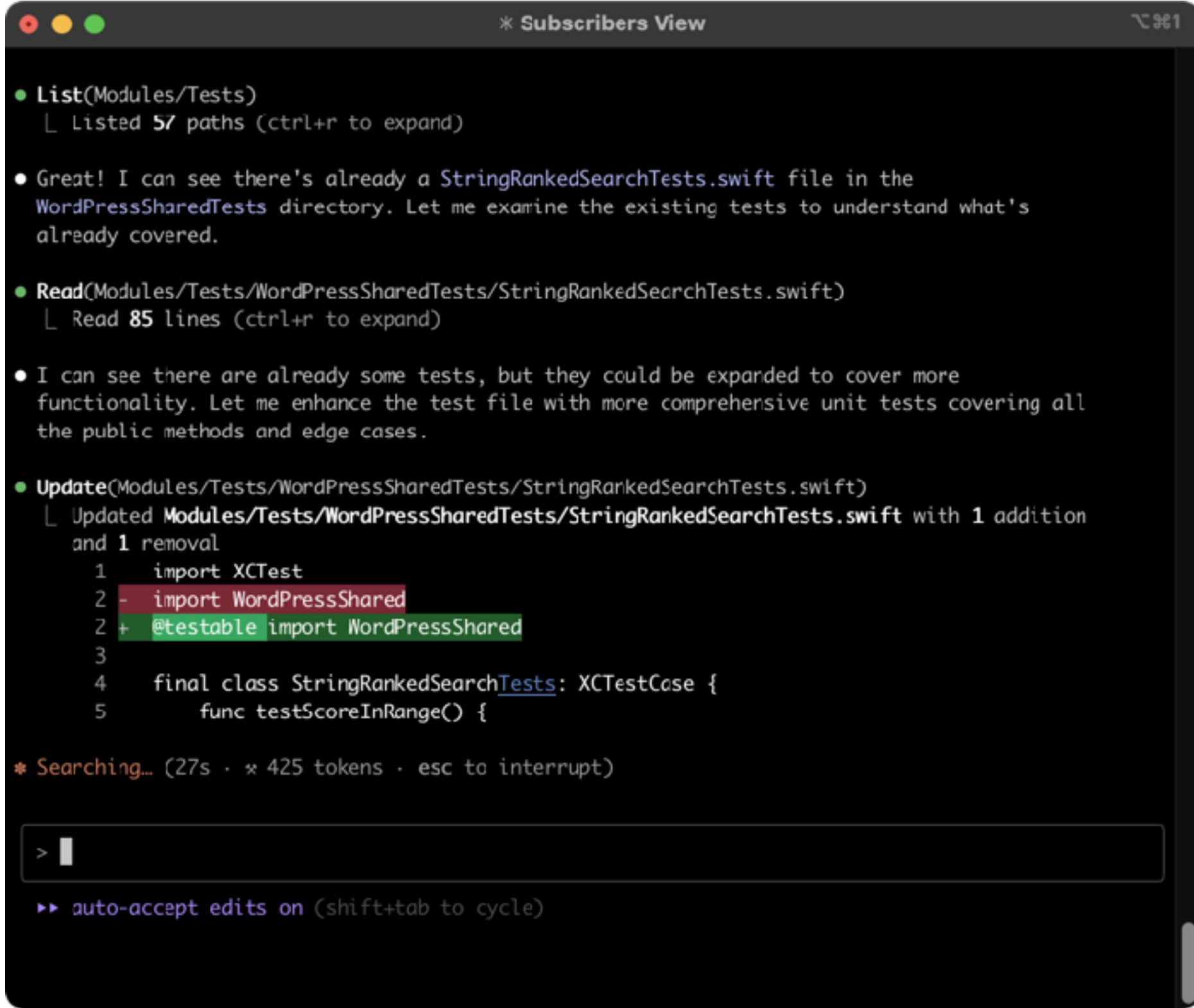


Interaction could be either in **IDE** (GitHub Copilot, Cursor, Codex, **Claude Code**, etc.) or **Browser** (**Claude**, ChatGPT, Gemini, etc.)

The spectrum of AI-assisted development

Option 3: Agentic. Model plans & executes across multiple steps autonomously

	Option 3: Agentic
What the model sees	In principle, the whole codebase
What you must specify	Goal, scope, and guardrails
Who assembles the context	The tool, by retrieval, and you can dictate the scope
Where it fails	When automatic retrieval picks wrong
What it's good at	Tasks that involves consistent changes across multiple files



```
* Subscribers View

• List(Modules/Tests)
  ↳ Listed 57 paths (ctrl+r to expand)

• Great! I can see there's already a StringRankedSearchTests.swift file in the
  WordPressSharedTests directory. Let me examine the existing tests to understand what's
  already covered.

• Read(Modules/Tests/WordPressSharedTests/StringRankedSearchTests.swift)
  ↳ Read 85 lines (ctrl+r to expand)

• I can see there are already some tests, but they could be expanded to cover more
  functionality. Let me enhance the test file with more comprehensive unit tests covering all
  the public methods and edge cases.

• Update(Modules/Tests/WordPressSharedTests/StringRankedSearchTests.swift)
  ↳ Updated Modules/Tests/WordPressSharedTests/StringRankedSearchTests.swift with 1 addition
  and 1 removal
    1 import XCTest
    2 - import WordPressShared
    2 + @testable import WordPressShared
    3
    4 final class StringRankedSearchTests: XCTestCase {
    5     func testScoreInRange() {

* Searching... (27s · × 425 tokens · esc to interrupt)

> |
>> auto-accept edits on (shift+tab to cycle)
```

Claude Code (terminal-based) (Other examples include Cursor, GitHub Copilot. Not available in *most enterprise-restricted environments*)

The spectrum of AI-assisted development

- What does this mean?
 - There are things that AI does better, and some that human engineer still prevails
 - Real and meaningful productivity gain are for a specific class of tasks, but not others

The spectrum of AI-assisted development

Where the industry is heading — and why it matters for your practice today

- We are obviously **moving towards more *agentic* workflows**
 - The tools are getting better
 - The models are getting more capable
- **The model** executes autonomously within a bounded scope, and **the engineer** reviews and steers at key decision points.

The spectrum of AI-assisted development

Where the industry is heading — and why it matters for your practice today

- In 2026
 - 55% of engineers regularly use AI agents; 63.5% among staff + engineers
 - 73% of engineering teams use AI coding tools daily (up from 41% in 2025)
- Cursor and Claude Code each reached 18% workplace adoption by January 2026

The spectrum of AI-assisted development

Where the industry is heading — and why it matters for your practice today

- While *adoption* is up, *trust* is shaking.
 - 84% of developers use or plan to use AI coding tools — up from 76% in 2024.
Trust in AI-generated output: 29% — down from 40%.
- Why?
 - As AI tools handle increasingly complex tasks — moving from single-function autocomplete to multi-file agentic changes
 - The surface area for *subtle errors* grows.

The spectrum of AI-assisted development

Where the industry is heading — and why it matters for your practice today

- The patterns for **making agents work reliably are also becoming clearer**
 - Most of the difficulty is in **knowing what to build, not how to build it**
 - People who write good prompts, manage context well, and verify output critically get **dramatically more value out of Agentic development**
 - Without the underlying skills, users are still able to get agent does something impressive, but they cannot diagnose why.

Why We Are Learning This Now

- Turning a vague intent into a precise specification is not a soft skill
 - It is a **technical skill** with measurable impact on output quality, and it is currently undervalued relative to its actual leverage
- We are not learning the skill of using a specific tool, but the skill of
 - knowing what to **specify**
 - what to **constrain**
 - what to **verify**

The engineer's new role: three responsibilities

- **Specifying intent:** writing instructions precise enough that the agent does what you mean, not just what you said
- **Evaluating trajectories:** assessing whether the agent is moving toward the right goal through the right path — not just whether the final output looks correct
- **Knowing when to take back control:** recognizing the moment when the agent has gone wrong and needs human intervention before it compounds an error

Why We Are Learning This Now

"But I'll just use *Claude Code (or any agentic system)* at work."

- Good — and these are the skills that make the “**Agentic workflow**” works.
- Every agentic tool ships **manual context controls**. These exist because automatic retrieval isn't sufficient. Using them well is manual context management.
- When the agent fails, you take over. Taking over is manual context management under time pressure.

@file mentions

CLAUDE.md

.cursorrules

"add to context"

context pinning

...

Why "trajectory" and not just "output"

- A single output can look correct while the overall trajectory is heading somewhere wrong
 - Example: an agent asked to fix a bug that correctly fixes the immediate error but introduces a security vulnerability two files away
- You cannot evaluate agentic engineering by looking only at what came out — you need to **understand the path the agent took**

Why "trajectory" and not just "output"

This framing applies to everything in the course

- Writing a prompt → you are specifying intent
- Testing a prompt system → you are evaluating whether your specification works
- Debugging AI-generated code → you are evaluating a proposed trajectory and deciding whether to accept it
- Building a RAG pipeline → you are specifying what the agent knows at each step
- The tool changes every week, but the engineer's responsibility does not.

Why We Are Learning This Now

Definition Made Clear

Vibe Coding	Prompt Engineering
An interaction model	A skill set
<i>What</i> you're doing	<i>How well</i> you do it
"I describe, model implements"	"I specify precisely, verify critically"
You can do it <i>badly</i>	This is what makes it <i>not-badly</i>

You can vibe code with bad prompts. You will get bad code

What you will be able to do by December

Recap – becoming the “AI-assisted Software Engineer”

- *Specify* intent precisely enough for an AI agent to execute it reliably
- *Evaluate AI output* — not just "does it look right" but "is it actually correct and trustworthy"
- Know *When to take back control* — override, correct, or choose not to use AI output
- *Build, test, and document* AI-integrated software that other engineers can understand and maintain
- *Collaborate* on a shared codebase where AI is a first-class participant

Course Logistics

Outline

- Course structure overview
- Final project overview
- Course resources overview
- Meet the instructors

Course structure overview

Lectures & Weekly Learning

- 16 weeks, Mondays, 10:10 - 12:00, from Sep 7 – Dec 21
 - **Holidays:** **Week 4** (Teacher's Day), **Week 8** (Retrocession Day)
 - **Special delivery weeks:** **Week 6** (async – both instructors traveling to conference), **Week 7** (online – time conflict traveling back)
- **Lecture each week:** ~60 min lecture + ~60 min lab in the same session
- **Self-paced module each week:** ~60 min completed during the week after class
 - There are 2-4 questions to complete for each week on E3

Course structure overview

Weekly Participation Grading

- **Labs:** all graded labs worth 1% each ($13 \text{ weeks} \times 1\% = 13\%$)
 - Week 3 worth 2%, as it is heavier and prepare everyone for their final project workflow
 - Labs **due by the end of day, Monday**
- **Self-paced module:** each week you are required to finish the reading, and respond to several questions based on the reading on E3
 - Self-paced module response **due by the end of the end of day, Sunday**

Course structure overview

Individual assignments gradings

- **Assignment 1 (due Week 5, 10%):** technique practice and prompt diagnosis
- **Assignment 2 (due Week 11, 15%):** prompt system design and test suite
- **Assignment 3 (due Week 15, 10%):** engineering decision portfolio — three decisions with actual artifacts
- **Self-paced module questions (worth 1% each week, 12 weeks $\times$ 1% = 12%)**
- **Course Experience Check-in surveys (4 surveys across semester, 4%)**

Course structure overview

Project assessments gradings

- **Three checkpoints:**
 - **CP1 (Week 6, 5%):** Problem framing, user research, initial prototype components, evaluation plan
 - **CP2 (Week 12, 15%):** Prototype build, test suite, peer review, collaboration sessions (Week 9 & 11)
 - **CP3 (Week 15, 12%):** Refinement, evaluation evidence, final report, prep for demo & presentation
- **Final presentation (Week 16):**
 - lightning talk + poster/demo + 5-minute video (Week 16, 5%)
- **Peer evaluation**
 - observable collaboration behaviors within each groups (submit at CP3, 2%)

Course structure overview

EMI (English as Medium of Instruction) Learning

- As an EMI course, this course will be taught in English
- Honestly speaking
 - As a non-native speaker, I also struggle with expressing myself in English from time to time
 - I also doubt if I will enjoy EMI as much if I am a college / graduate student now
- As the instructor of this course, I'm also learning how I could be a better teacher.

Course structure overview

EMI (English as Medium of Instruction) Learning

- **Practice in this course**
 - Lectures & Self-paced Module: 99% in English
 - Assignments, labs, all submissions: required in English, unless otherwise noted
- **Communicating with course staffs:** Mandarin + English, whichever you're comfortable
- We're here to help you learn throughout the course of this semester
 - If you have questions, both instructors and all the TAs are happy to help

Final Project

“Collaboration Tools for Software Engineering and Academic Teams”

- Each group will consists of four students
- You and your team will build
 - A working tool that reduces a real collaboration pain point for software engineering or academic teams, and provide evidence that it actually works”
- This is a collaboration-intensive course
 - We want to treat this as an opportunity for you to **actually collaborate on a software engineering project**
 - So, we are requiring multiple components to be done during the semester...

Final Project

Collaboration Framework

- **Version Control** using Git on Github
 - You will need to submit **Pull Requests** and get approved in your team
- Rotating **Code Steward** as engineering as designated PR reviewer
- You will keep a file (**DECISIONS.md**) to document significant decision — architectural, AI integration, process, or user experience, etc.
- Three Mandatory Whole-Team Collaboration Sessions (outside of class)
- Collaboration outcomes will be verified at checkpoints

Final Project

Collaboration Tools for Software Engineering and Academic Teams

- What we **are** grading
 - Problem specificity and evidence
 - AI integration judgment
 - Evaluation honesty
 - Documentation usefulness
 - **Collaboration process quality**

Final Project

Collaboration Tools for Software Engineering and Academic Teams

- What we are **not** grading
 - Novelty or technical sophistication
 - Visual polish
- Whether your final outcome incorporates AI usage
 - You will **not** be penalized for building something without AI if the problem does not warrant it

Final Project

The prompt engineering log

- We encourage each student to keep a *prompt engineering log*
- A running document you maintain throughout the semester
- Every significant AI interaction: what you prompted, what it produced, what you changed and why
- Individual, not team — your personal record of your engineering decisions with AI
- This will be heavily referenced in **Assignment 3**

Course relevant tools

Platforms and tools

- All course materials will be stored on **E3**, including course slides, videos for self-paced modules, supplementary materials, etc.
- All announcements will be posted through **E3** (and **email notification** will come through), please **pay the most attention to these announcements**
- The **course website** will be where you can find all information about this course.

Course relevant tools

Communications

- We will prioritize checking emails sent to the **course mailing list (TBA)**
 - **Please note:** we will **NOT** be checking E3's email inbox regularly. Sending emails to us on E3 will risk messages being unnoticed by the course staffs.
- **E3's discussion board** will be open for discussion on different components of the course, including assignment submissions (individual, teams, labs), course content, and general discussion.
- Please allow **48 hours** for the course staffs to get to your messages

Course relevant tools

Ideal Practices

- I would suggest —
 - Check the [course website](#) frequently, and use it to navigate to different components in the course
 - Check your [email](#) and the announcements on [E3](#) for the latest announcements
 - Email the [course staff mailing list](#) to get the fastest response

Course Resources

AI model for the class

- For this year's course, we will be using ***Claude*** as the model that our course content is based on
 - However, we know that you might already have a model that you prefer, and we welcome you to use whichever model you feel the most confident / comfortable to work with
 - Most of the materials would be model-agnostic, with some using Claude-depend APIs.

Course Resources

The AI tool fund for CSIC30216

- We will provide an amount of quota for using AI tool for students in this class
 - We are currently finalizing the plan for a given API budget that we will provide to the students of this course.
- The API quota will be activated from October
- The goal of an API budget is to support people working on course related work, specifically on the *Final Project*
- More information to be provided in the coming weeks...

Meet the Instructors

- **Dennis Wang 王邦任**

Assistant Professor of Computer Science,
National Yang Ming Chiao Tung University
(Started August, 2025)

- **Education:**

Ph.D. in Informatics

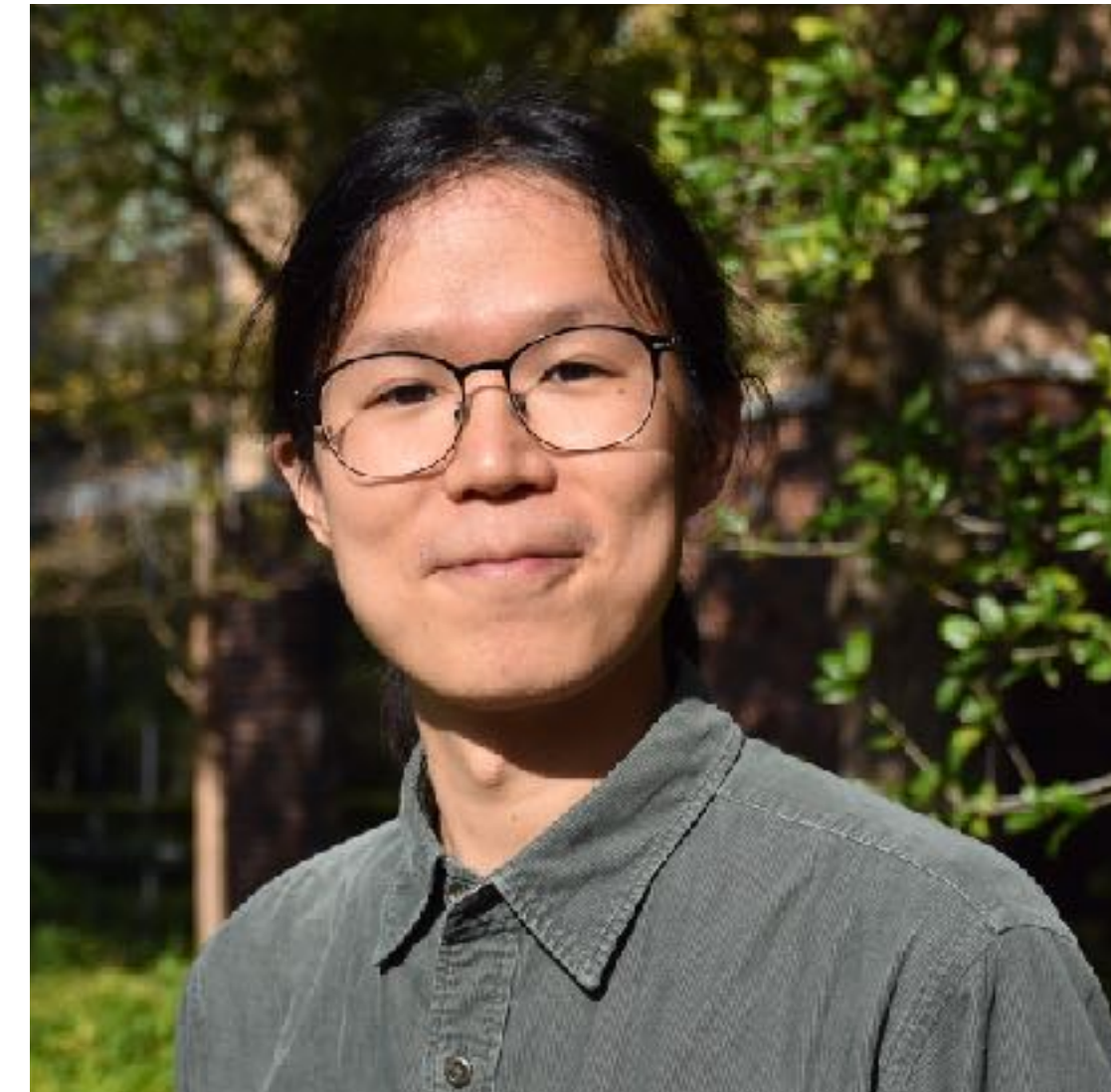
University of California, Irvine, USA

M.S. in Computer Science,

University of Illinois at Urbana Champaign, USA

B.S. in Computer Science,

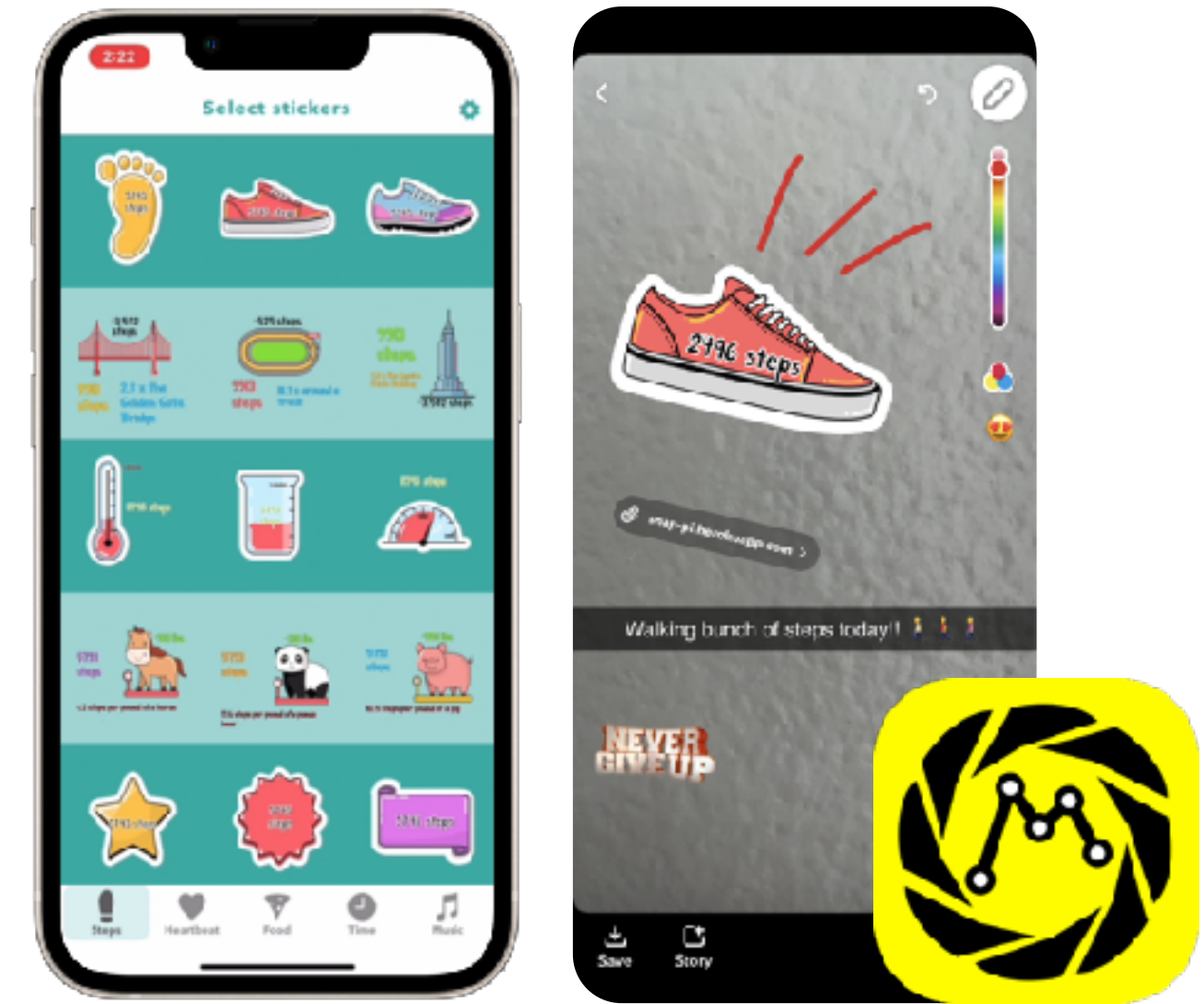
National Chengchi University, Taiwan



**Social
Computing for
Well-being**

Meet the Instructors

- My Research Area is *Human-Computer Interaction*
- I lead the research work in **Social Computing for Well-being Lab**
(社群運算與福祉實驗室)
- My team and I study how we could design **social technology** to improve people's well-being
 - Our research looks at — **Understanding and Designing For Positive Online Social Norms**

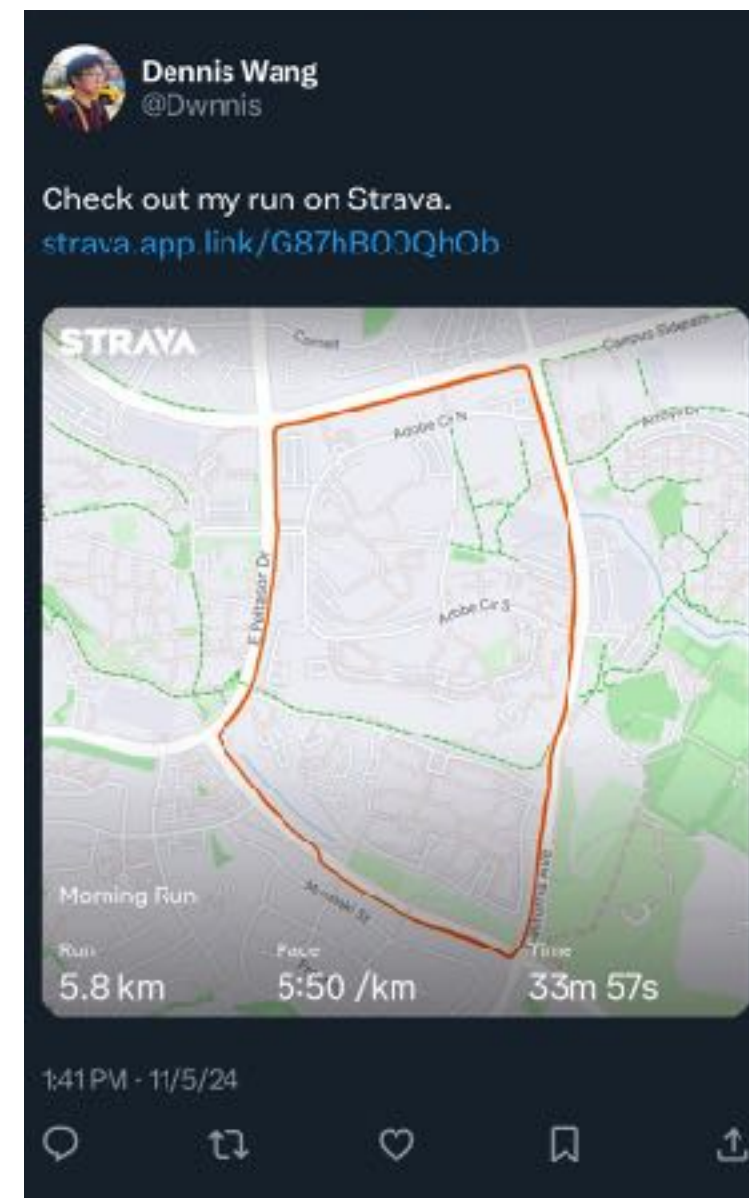


Meet the Instructors

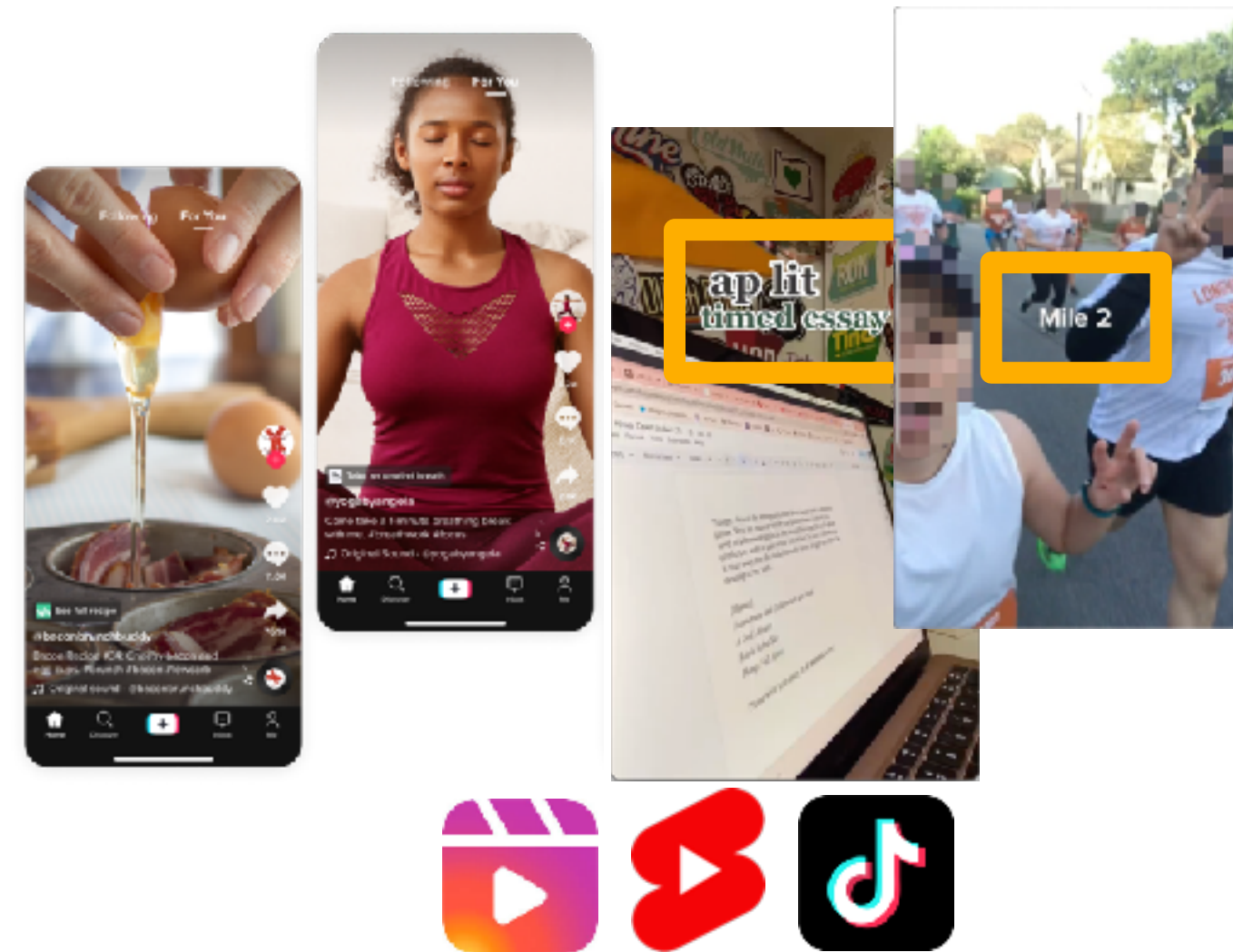
Understanding and Designing For Positive Online Social Norms



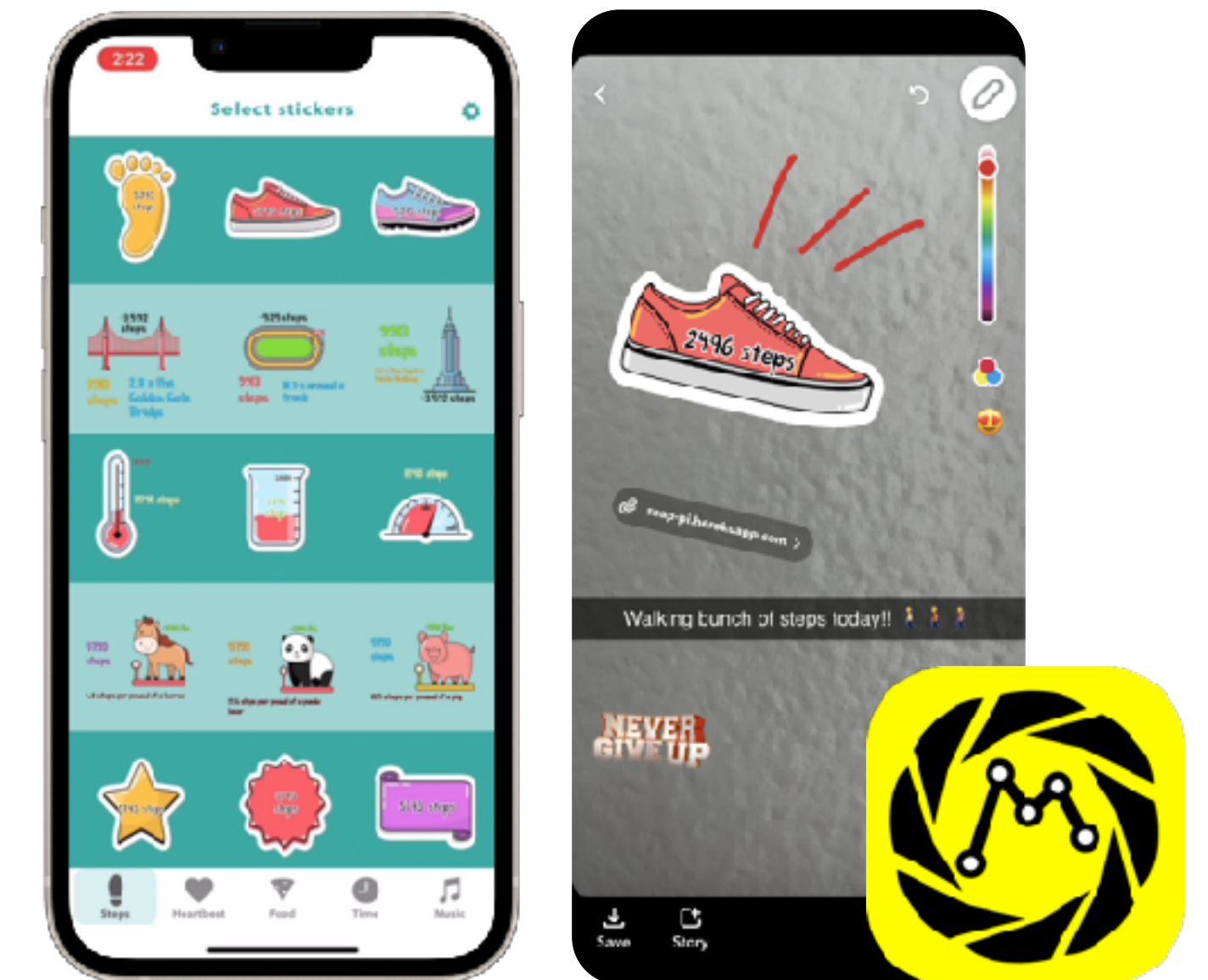
 Strava



 Twitter



How people incorporate
self-tracking data / information
in **Short-form Videos**?



How do we design **authoring tool**
for self-tracking information that
help aligning with norm of
Ephemeral Social Media?

The self-paced module for this week

- **Topic:** *The history of language models*
 - word embeddings, transformer architecture (intuition only), pre-training vs instruction tuning, current model landscape
- You just saw a model doing something sophisticated — the self-paced module answers "where did this come from and what makes it possible?"
- ~60 minutes, complete before next Monday
- Completion = finishing the reading, and responded to the questions on E3

The self-paced module for this week

What the self-paced module is not

- It is **NOT** optional reading
- It is preparation for the next lecture — Week 2 assumes you have done it
- Each week's module is designed to deepen what happened in class, not repeat it

Today's lab: Experiencing the intent-output gap

- **Goal of the lab:** make you feel the gap between what you meant and what the model produced
 - No code, no API keys, no setup required
- **The core activity:**
 - write a prompt → observe the output → notice the gap → revise → observe again

The task

- A Python function is provided on the board — doing something non-obvious
 - Your task: write a prompt that gets the model to explain what this code does
 - That is the whole instruction — "explain what this code does"

Explain what this code does

```
def mystery(items):  
    if not items:  
        return []  
    pivot = items[0]  
    smaller = [x for x in items[1:] if x <= pivot]  
    larger = [x for x in items[1:] if x > pivot]  
    return mystery(smaller) + [pivot] + mystery(larger)
```

Part 1: write your first prompt (10 minutes)

- Write a prompt asking the model to explain this function
 - Write it the way you naturally would — do not overthink it
 - Do not discuss with neighbors — your instinct is the data
 - Write it down (paper or your laptop) — you will need it later

Part 2: run your prompts (10 minutes)

- Run the prompts
 - For each output, ask yourself one question: "What did the model interpret differently from what you intended?"
 - Not: was this good or bad? Not: did you like it?
 - Specifically: what decision did the model make that you did not make explicitly?
- We invite a few of your prompts to run live — any volunteers?

What to look for in the outputs

- Did the model choose an audience without being told who would read the explanation?
- Did it choose a level of technical detail without being told the reader's background?
- Did it explain what the function does mechanically, or what it is for?
- Did it mention the recursion explicitly or describe the behaviour without naming the technique?
- Did it notice the empty list edge case — and if so, did it explain why that matters?

Part 3: revise your prompt (10 minutes, individual)

- Write a second version of your prompt
- Rule:
 - **change only one thing**
- Not a complete rewrite — identify the single most important gap and address it

Part 4: run your prompts (10 minutes)

- Run the prompts
 - For each output, ask yourself one question: "What did the model interpret differently from what you intended?"
 - Not: was this good or bad? Not: did you like it?
 - Specifically: what decision did the model make that you did not make explicitly?
- The same three or four prompts from Part 2, revised versions run live
- For each pair (original + revision): did the gap narrow? Did a new gap appear?

What you probably noticed

- "Explain" was doing an enormous amount of hidden work — it contained at least five implicit decisions
 - Making one decision explicit changed the output meaningfully
 - But making one decision explicit sometimes revealed another implicit decision you had not noticed
- The model is not misunderstanding you — it is interpreting you, based on what you wrote

Prompt engineering log: first entry

- Write your first prompt engineering log entry — four fields:
 - What you asked the model to do (one sentence — your original prompt)
 - What it produced (one sentence — not the full output, just what kind of thing it was)
 - What you decided to change and why (one to two sentences — what gap you identified and what you did about it)
 - What this tells you about the gap between intent and output (one sentence)
- This is your first entry in a log you will maintain all semester

Lab: Wrapping Up

- The model interpreting your instruction differently than you intended
 - What you just experienced is the core challenge of agentic engineering
- "Explain what this code does" seems like a complete instruction
 - **It is not** — it contains at least five implicit decisions you delegated to the model without knowing it
- Your job as an engineer:
 - to make those decisions explicitly, and to evaluate whether the model made them the way you would have

Lab: Wrapping Up

- Submission for the week:
 - Your prompt engineering log

What to do before next Monday

- ☐ Complete the **self-paced module**: history of language models (~60 min)
 - This will be uploaded today on **the course website (and E3)**.
 - This is **not graded** for the first week, but you might want to get familiar with the format.
- ☐ Start your **prompt engineering log** — first entry is from today's lab
- ☐ Submit your **prompt engineering log** from today's lab
 - This is **also not graded** for the first week, but doing it will help you get familiar with the format.
- ☐ **Start to form your project team** — teams of 4 due before Week 3

Week 2 preview

- **Topic:** How LLMs actually work
- We will go from "the model predicts the next token" to what that actually means for how you prompt it
- The Week 2 lab is an API exploration — bring a laptop with the Anthropic SDK installed
- SDK installation instructions posted on the course portal today

Closing note: one thing to remember from today

- The engineer's job in the agentic era is not to use AI tools, but to **specify intent, evaluate trajectories, and know when to take back control**
 - Everything else in this course is in service of doing those three things better

Learning Objectives

- ✓ Describe the engineer's three responsibilities in the agentic era: specifying intent, evaluating trajectories, and knowing when to take back control
- ✓ Understand the course structure, components, expectations, and resources
- ✓ Identify at least one decision the model made without being asked during the demo and evaluate whether you would have made the same choice
- ✓ Write your first prompt engineering log entry based on today's lab

See you next week

- *Week 2: How LLMs actually work*
- Monday Sep 14, same room, same time
- Self-paced module due before class

CSIC30216

AI-in-the-Loop in Software Project Cycle

AI 導入軟體專案流程

Week 1: Introduction to AI-assisted
software engineering

Instructors:
Dennis Wang 王邦任
Yu-Chun (Grace) Yen 顏羽君

Date: 2026-09-07